

Proteção da `main` no GitHub — Fundamentos, Configuração, Verificação e Correção por CLI

Contexto: você acabou de ativar a proteção da `main` com PR obrigatório, checks `lint/build/test`, exigência de branch atualizada, sem force push/deleções e sem bypass por admins. Este guia explica **por que** isso é importante, **o que** cada opção faz, **como verificar** e **como corrigir** via `gh api` — incluindo o que deu errado e por quê na primeira tentativa.

1) Por que proteger a `main`?

- **Estabilidade:** impede que mudanças não revisadas quebrem o projeto em produção ou nos ambientes de ensino.
- **Qualidade de código:** força revisão por pares e execução do pipeline (lint/build/test), reduzindo regressões.
- **Rastreabilidade e auditoria:** toda alteração passa por PR, com histórico claro de discussões, commits e aprovações.
- **Educação para o mercado:** os alunos vivenciam um fluxo profissional (PR, revisão, CI, políticas de branch).

2) O que cada opção garante (UI → API)

A tabela resume as opções ativadas, o **risco mitigado**, quando **ajustar**, e o campo correspondente na **API**.

Opção (UI)	Responsabilidade / Risco mitigado	Quando ajustar	Campo API (v3)
Require a pull request before merging	Bloqueia merge direto na <code>main</code> ; exige revisão e CI	Pode subir de 1→2 aprovações quando a turma crescer	<code>required_pull_request_reviews.*</code> + PR obrigatório
Require status checks to pass before merging (lint/build/test)	Garante que CI está verde antes do merge	Adicionar mais contexts (ex.: <code>e2e</code> , <code>audit</code>) conforme amadurecer	<code>required_status_checks.contexts</code>

Opção (UI)	Responsabilidade / Risco mitigado	Quando ajustar	Campo API (v3)
Require branches to be up to date	O PR precisa ser re-testado contra a <code>main</code> mais recente	Desligar só em casos raros de filas longas	<code>required_status_checks.strict=true</code>
Disallow force pushes	Evita reescrever histórico (perda de auditoria)	Nunca ativar em sala/aula	<code>allow_force_pushes=false</code>
Disallow deletions	Impede apagar a <code>main</code> acidentalmente	Sempre manter desligado	<code>allow_deletions=false</code>
Do not allow bypass (admins)	Nem admins pulam regras por engano	Em orgs com emergência real, decidir fora do contexto didático	<code>enforce_admins=true</code>
Required approving reviews (1)	Exige ao menos 1 aprovação	Para turmas maiores, usar 2	<code>required_pull_request_reviews.required_approving_</code>
Dismiss stale reviews	Novos commits invalidam aprovações antigas	Sempre que houver push novo ao PR	<code>required_pull_request_reviews.dismiss_stale_review</code>
Require approval of the most recent push	Garante que o último commit foi revisado	Essencial para evitar “aprova e muda depois”	<code>required_pull_request_reviews.require_last_push_a</code>
Require conversation resolution	Obriga resolver todos os comentários antes do merge	Excelente didática de revisão	<code>required_conversation_resolution=true</code>
Require linear history	Força histórico sem merge commits (preferir squash)	Ativar junto de “Squash and merge”	<code>required_linear_history=true</code>
Require review from Code Owners	Obriga donos de área a aprovarem	Ativar depois de subir <code>/.github/CODEOWNERS</code>	<code>required_pull_request_reviews.require_code_owner</code>

Dica pedagógica: manter **Squash and merge** no repositório (Settings → Merge button) ajuda a manter o histórico limpo quando `required_linear_history=true`.

3) Fluxo recomendado para a turma

1. **Abrir branch feature** → commits pequenos e descritivos.
 2. **Abrir PR** → CI roda (`lint/build/test`).
 3. **Correções até CI verde** → pedir revisão do colega.
 4. **Aprovação (1+) + conversas resolvidas + branch up-to-date.**
 5. **Squash and merge** → histórico limpo e rastreável.
-

4) Verificação por CLI (auditoria rápida)

Após configurar, valide com:

```
# Resumo objetivo
gh api repos/:owner/:repo/branches/main/protection | jq
'{strict: .required_status_checks.strict,
 contexts: .required_status_checks.contexts,
 reviews: .required_pull_request_reviews,
 enforce_admins: .enforce_admins.enabled,
 linear: .required_linear_history.enabled,
 conversations: .required_conversation_resolution.enabled}'
```

Esperado após o seu ajuste: - `strict: true` e `contexts: ["lint","build","test"]` - reviews: `required_approving_review_count: 1`, `dismiss_stale_reviews: true`, `require_last_push_approval: true` - `enforce_admins: true`, `linear: true`, `conversations: true`

5) Correção por CLI (JSON robusto)

A forma **mais estável** é enviar um **corpo JSON** (evita problemas com `zsh` e campos aninhados):

```
# Base (sem CODEOWNERS obrigatório)
gh api
-X PUT
-H "Accept: application/vnd.github+json"
repos/:owner/:repo/branches/main/protection
--input - <<'JSON'
{
  "required_status_checks": {
    "strict": true,
    "contexts": ["lint", "build", "test"]
  },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 1,
```

```

    "dismiss_stale_reviews": true,
    "require_last_push_approval": true,
    "require_code_owner_reviews": false
  },
  "required_conversation_resolution": true,
  "required_linear_history": true,
  "allow_force_pushes": false,
  "allow_deletions": false,
  "restrictions": null
}
JSON

```

Quando `/ .github/CODEOWNERS` existir e você desejar exigir seus reviews:

```

# Habilitar Code Owners como obrigatórios
gh api -X PUT -H "Accept: application/vnd.github+json"
  repos/:owner/:repo/branches/main/protection
  --input - <<'JSON'
{
  "required_status_checks": { "strict": true, "contexts":
["lint","build","test"] },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 1,
    "dismiss_stale_reviews": true,
    "require_last_push_approval": true,
    "require_code_owner_reviews": true
  },
  "required_conversation_resolution": true,
  "required_linear_history": true,
  "allow_force_pushes": false,
  "allow_deletions": false,
  "restrictions": null
}
JSON

```

6) O que deu errado antes? E por quê?

- **Sintoma:** após enviar com `-F` (form-data), somente `required_status_checks` mudou; os demais ficaram no padrão (`false` / `0`).
- **Causas prováveis:**
- **zsh e colchetes:** parâmetros como `required_status_checks.contexts[]=lint` sofrem *globbing*. Sem aspas/escape, o shell tenta expandir `[]` e falha (`no matches found`).
- **Campos aninhados em form-data:** a API do GitHub nem sempre interpreta corretamente estruturas profundas quando chegam como form-data; alguns subcampos são **ignorados** ou sobrescritos por defaults.

- **Correção:** usar **JSON** (`--input - <<'JSON' ... JSON`) contorna o *globbing* e garante a estrutura hierárquica correta. Alternativas paliativas: **aspas** nos campos com `[]`, **escapar** colchetes (`\[\]`) ou `noglob` — mas **JSON** segue sendo o caminho robusto.

7) O que deu certo?

- `required_status_checks` aplicou com `strict:true` e contexts `[lint, build, test]` (já havia contexts porque a CI rodou pelo menos uma vez).
- Depois, com JSON, todos os campos ficaram conforme esperado: `enforce_admins`, `required_approving_review_count`, `dismiss_stale_reviews`, `require_last_push_approval`, `required_conversation_resolution`, `required_linear_history`.

8) Checklist de Auditoria (para README)

- [x] PR obrigatório
- [x] Status checks obrigatórios: lint, build, test (strict: true)
- [x] Branch up-to-date exigida
- [x] Enforce admins (sem bypass)
- [x] 1 aprovação + dismiss stale + last push approval
- [x] Conversation resolution
- [x] Linear history
- [] CODEOWNERS ativo
- [] Require review from Code Owners

9) Troubleshooting rápido

- **Contexts não aparecem:** rode a CI ao menos uma vez (abra um PR com `ci.yml` já presente). Sem execução, a UI não lista `lint/build/test`.
- **Merge bloqueado após ativar Code Owners:** suba o arquivo `/.github/CODEOWNERS` e confirme se os paths batem com os arquivos alterados no PR.
- **Admins ainda conseguem mesclar:** verifique `enforce_admins.enabled` no JSON de proteção; deve estar `true`.
- **Histórico com merges** mesmo com linear history: garanta que o botão de merge está configurado para **Squash and merge** e/ou reforce no time.

10) Script utilitário (opcional)

Crie `scripts/protect_main.sh`:

```
#!/usr/bin/env bash
set -euo pipefail
```

```

REPO="${1:?Uso: $0 owner/repo}"

read -r -d '' BODY <<'JSON'
{
  "required_status_checks": { "strict": true, "contexts":
["lint","build","test"] },
  "enforce_admins": true,
  "required_pull_request_reviews": {
    "required_approving_review_count": 1,
    "dismiss_stale_reviews": true,
    "require_last_push_approval": true,
    "require_code_owner_reviews": false
  },
  "required_conversation_resolution": true,
  "required_linear_history": true,
  "allow_force_pushes": false,
  "allow_deletions": false,
  "restrictions": null
}
JSON

echo "Aplicando proteção na main de $REPO..."
echo "$BODY" | gh api -X PUT -H "Accept: application/vnd.github+json"
  repos/$REPO/branches/main/protection --input - >/dev/null

echo "Estado atual:"
gh api repos/$REPO/branches/main/protection | jq
'{"strict: .required_status_checks.strict,
  contexts: .required_status_checks.contexts,
  reviews: .required_pull_request_reviews,
  enforce_admins: .enforce_admins.enabled,
  linear: .required_linear_history.enabled,
  conversations: .required_conversation_resolution.enabled}'

```

Execute com: `bash scripts/protect_main.sh evertonfoz/dsc-2025-2-aurora-platform`

Conclusão

Você consolidou um **padrão profissional de governança** na `main`: PR obrigatório, checks verdes, branch atualizada, sem force push/deleções e sem bypass por admins — além de regras de revisão que reforçam a qualidade do aprendizado (stale dismiss, last push approval, conversas resolvidas e histórico linear). O próximo passo natural é **adicionar CODEOWNERS** e, então, **exigir revisão dos donos de área**, alinhando responsabilidade técnica à estrutura do microserviço.